Numerical Computation in C: Fundamentals - Chapter 0 - Numerical Analysis by Timothy Sauer.

2. Computer Representation of Real Numbers

Rounding errors are inevitable when finite-precision computer memory locations are used to represent real, infinite precision numbers. Although we would hope that small errors made during a long calculation have only a minor effect on the answer, this turns out to be wishful thinking in many cases.

IEEE Floating Point Standards

Real numbers are represented on computers using the IEEE 754 floating-point standard. This format breaks a number into three parts:

Precision Levels

1. **Single Precision (32 bits):**
   - **Sign bit (1 bit):** Determines if the number is positive (`0`) or negative (`1`).
   - **Exponent (8 bits):** Stores the power of 2, shifted by a bias of 127 to allow representation of negative exponents.
   - **Mantissa (23 bits):** Contains the significant digits of the number in binary, normalized to a format starting with `1.`.
2. **Double Precision (64 bits):**
   - **Sign bit (1 bit):** Same as single precision.
   - **Exponent (11 bits):** Stores the power of 2, shifted by a bias of 1023.
   - **Mantissa (52 bits):** Contains the significant digits of the number in binary.

Floating-Point Representation

The general form of a floating-point number is:

$$\text{Value} = (-1)^{\text{sign}} \times 1.\text{mantissa} \times 2^{\text{exponent - bias}}.$$

Demonstrating Floating-Point Precision Issues with `0.1 + 0.2 == 0.3`

Here is a C program to show why `0.1 + 0.2 == 0.3` evaluates to `false`:

```
# include <stdio.h>
# include <stdbool.h>

int main() {
```

```
    double a = 0.1;
    double b = 0.2;
    double c = 0.3;

    // Direct comparison
    if (a + b == c) {
        printf("0.1 + 0.2 == 0.3 is True\n");
    } else {
        printf("0.1 + 0.2 == 0.3 is False\n");
    }

    // Show the actual values
    printf("a + b = %.17f\n", a + b);
    printf("c = %.17f\n", c);

    return 0;
}
```

Explanation of the Code

1. **What Happens in Floating-Point Arithmetic?**
   - Floating-point numbers (like `0.1`, `0.2`, and `0.3`) are represented in binary with finite precision (52 bits for the mantissa in double precision).
   - Many decimal fractions, such as `0.1` and `0.2`, do not have exact binary representations. Instead, they are approximated:
     - $0.1_{10} \approx 0.0001100110011\ldots_2$ (repeating).
     - $0.2_{10} \approx 0.001100110011\ldots_2$.
   - Adding these approximations does not yield an exact representation for `0.3`.
2. **Why is the Comparison False?**
   - When `a + b` is computed, the result is an approximation close to but not exactly `0.3`.
   - Comparing `a + b == c` fails because the internal binary representation of `a + b` differs from `c`.
3. **Output of the Program:** The output will be:

   ```
   0.1 + 0.2 == 0.3 is False
   a + b = 0.30000000000000004
   c = 0.30000000000000000
   ```

   This shows that the sum $0.1 + 0.2$ is slightly greater than 0.3 due to rounding errors in floating-point arithmetic.

Understanding Machine Epsilon ($\varepsilon_{\text{mach}}$)

1. **Definition:**
   Machine epsilon is the smallest number that can be added to 1.0 such that the result is distinguishable from 1.0 in the given floating-point representation.

2. **Connection to the Example:**
   The error in representing 0.1, 0.2, and 0.3 arises because these numbers are approximated within the precision limits of double-precision floating-point representation. This leads to rounding errors, and `0.1 + 0.2` being slightly off from `0.3`.

3. **How to Compute Machine Epsilon in C:**
   Here's a snippet to calculate $\varepsilon_{\text{mach}}$:

   ```
   #include <stdio.h>

   int main() {
       double epsilon = 1.0;

       while (1.0 + epsilon != 1.0) {
           epsilon /= 2.0;
       }

       epsilon *= 2.0; // Last value before the condition failed
       printf("Machine epsilon: %.17f\n", epsilon);

       return 0;
   }
   ```

   Output:

   ```
   Machine epsilon: 0.00000000000000022
   ```

   This demonstrates the precision limit of double-precision floating-point numbers, which is $2^{-52} \approx 2.22 \times 10^{-16}$.

Key Takeaways:

- Floating-point numbers are approximations of real numbers, leading to errors in calculations.
- Machine epsilon quantifies the smallest meaningful difference in floating-point arithmetic.

- When comparing floating-point numbers, avoid direct comparisons (e.g., `a + b == c`) and instead check if the difference is smaller than a tolerance:

```
if (fabs((a + b) - c) < 1e-15) {
    printf("Close enough!\n");
}
```

Key Concepts

1. **Machine Epsilon ($\varepsilon_{\text{mach}}$):**
   - The smallest number that, when added to 1.0, results in a number distinguishable from 1.0 in the given precision.
   - For double precision: $\varepsilon_{\text{mach}} = 2^{-52} \approx 2.22 \times 10^{-16}$.
2. **Rounding Error:**
   - The difference between the exact number and its floating-point representation.
   - For 9.4_{10}, the exact binary has infinite bits, but double precision rounds the last bit of the mantissa, introducing a small error.
3. **Bias:**
   The exponent is stored as an unsigned integer with a bias to handle negative exponents. In double precision, the bias is 1023.

Rounding Techniques

1. **Chopping**: Discard extra bits without modification.
2. **Rounding**: Round up or down based on the next bit.

Numerical Differentiation in C:

The notes are based on the Chapter 5 of **Numerial Analysis** book by *Timothy Sauer.*

The main problem of computational calculus is to compute derivatives and integrals of functions. Numerical differentiation is the process of approximating the derivatives of a function using discrete data points. To begin, we develop finite difference formulas for approximating derivatives. In some cases, that is the goal of the calculation.

Introduction

The derivative of a function $f(x)$ is defined as:

$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h},$$

where h is a small increment. In numerical differentiation, h cannot approach zero due to limitations in machine precision, which leads to truncation and rounding errors.

Why Numerical Differentiation?

- Useful for functions defined by discrete data points or complex mathematical expressions.
- Essential in engineering and science for analyzing changes in data.

Finite Difference Formulas

Finite difference methods approximate derivatives using Taylor series expansions. These methods are categorized as forward, backward, or centered differences.

1. Forward Difference Formula

Using Taylor's theorem if f is twice continuously differentiable, then:

$$f(x+h) = f(x) + hf'(x) + \frac{h^2}{2}f''(c),$$

we derive the forward difference formula:

$$f'(x) = \frac{f(x+h)-f(x)}{h} - \frac{h}{2}f''(c).$$

where c is between x and $x+h$.

$$f'(x) \approx \frac{f(x+h)-f(x)}{h}$$

treating the last term in equation above as error. Because the error made by the approximation is proportional to the increment h, we can make the error small by making h small. The *two-point-forward-difference* formula is a first-order method for approximating the first derivative. In general, if the error is $O(h^n)$, we call the formula an order n approximation.

C Example

```
# include <stdio.h>
# include <math.h>

// Function to approximate
double f(double x) {
    return log(x); // Example: natural logarithm
}

// Forward Difference Approximation
double forward_difference(double x, double h) {
    return (f(x + h) - f(x)) / h;
}

int main() {
```

```
    double x = 1.0;
    double h = 0.01;

    printf("Forward Difference Approximation at x = %.2f: %.6f\n", x, forward_difference(x, h));
    return 0;
}
```

To run it in terminal: use-

```
gcc numerical_diff.c -o numerical_diff -lm
./numerical_diff
```

2. Centered Difference Formula

Using Taylor's expansions:

$$f(x+h) = f(x) + hf'(x) + \frac{h^2}{2}f''(x) + \frac{h^3}{6}f'''(c_1),$$
$$f(x-h) = f(x) - hf'(x) + \frac{h^2}{2}f''(x) - \frac{h^3}{6}f'''(c_2),$$

we subtract to derive:

$$f'(x) \approx \frac{f(x+h)-f(x-h)}{2h} - \frac{h^2}{6}f'''(c).$$

Error: Proportional to h^2, making it a second-order method.

C Example

```
// Centered Difference Approximation
double centered_difference(double x, double h) {
    return (f(x + h) - f(x - h)) / (2 * h);
}

int main() {
    double x = 1.0;
    double h = 0.01;

    printf("Centered Difference Approximation at x = %.2f: %.6f\n", x, centered_difference(x, h))
    return 0;
}
```

3. Higher-Order Derivatives

For second derivatives, the formula is:

$$f''(x) \approx \frac{f(x+h)-2f(x)+f(x-h)}{h^2}.$$

C Example

```
// Second Derivative Approximation
double second_derivative(double x, double h) {
    return (f(x + h) - 2 * f(x) + f(x - h)) / (h * h);
}

int main() {
    double x = 1.0;
    double h = 0.01;

    printf("Second Derivative Approximation at x = %.2f: %.6f\n", x, second_derivative(x, h));
    return 0;
}
```

Rounding Errors in Numerical Differentiation

Rounding errors occur due to finite precision in computer arithmetic. For small h, subtracting nearly equal numbers leads to significant loss of precision.

The total error is:

$E(h) = \text{Truncation Error} + \text{Rounding Error}$, where truncation error decreases with smaller h, but rounding error increases.

Practical Examples

Example 1: Compute $f'(x)$ for $f(x) = e^x$ at $x = 0$

Code:

```
# include <stdio.h>
# include <math.h>

double f(double x) {
    return exp(x);
}

int main() {
    double x = 0.0;
    double h = 0.001;

    printf("Forward Difference: %.8f\n", forward_difference(x, h));
    printf("Centered Difference: %.8f\n", centered_difference(x, h));
```

```
    return 0;
}
```

Output:

```
Forward Difference: 1.00050017
Centered Difference: 1.00000017
```

Optimal Step Size

To minimize errors, the optimal h is:

$h_{opt} = \left(\frac{3\epsilon_{\text{mach}}}{M}\right)^{1/3}$, where ϵ_{mach} is machine epsilon, and M is an estimate of $f'''(x)$.

Example 2: Analyze Errors

Compute derivatives for $h = 10^{-1}$ to 10^{-9} and observe error trends.

Solution:

```
# include <stdio.h>
# include <math.h>

// Function to approximate
double f(double x) {
    return log(x); // Natural logarithm
}

// Exact derivative of f(x)
double exact_derivative(double x) {
    return 1 / x; // Derivative of ln(x)
}

// Third derivative of f(x) (used for h_opt calculation)
double third_derivative(double x) {
    return -2 / (x * x * x); // Third derivative of ln(x)
}

// Forward Difference Approximation
double forward_difference(double x, double h) {
    return (f(x + h) - f(x)) / h;
}

// Compute h_opt
double compute_h_opt(double x) {
    double epsilon = pow(2, -52); // Machine epsilon for double precision
```

```
    double M = fabs(third_derivative(x)); // Estimate of f'''(x)
    return pow((3 * epsilon) / M, 1.0 / 3.0);
}

int main() {
    double x = 1.0; // Point of evaluation
    double h_values[] = {1e-1, 1e-2, 1e-3, 1e-4, 1e-5, 1e-6, 1e-7, 1e-8, 1e-9};
    int num_h = sizeof(h_values) / sizeof(h_values[0]);

    printf("Derivative approximations for f(x) = ln(x) at x = %.2f:\n", x);
    printf("h\t\tApproximation\tExact\t\tError\n");
    printf("----------------------------------------------------\n");

    for (int i = 0; i < num_h; i++) {
        double h = h_values[i];
        double approx = forward_difference(x, h);
        double exact = exact_derivative(x);
        double error = fabs(approx - exact);
        printf("%.1e\t%.6f\t%.6f\t%.6e\n", h, approx, exact, error);
    }

    // Compute h_opt
    double h_opt = compute_h_opt(x);
    printf("\nOptimal step size (h_opt): %.6e\n", h_opt);

    return 0;
}
```

Conclusion

Numerical differentiation is a powerful tool for approximating derivatives when symbolic methods fail. Understanding finite difference methods and managing rounding errors is crucial for accurate results. Experiment with the provided examples to deepen your understanding.

```
Start coding or generate with AI.
```

Double-click (or enter) to edit

```
Start coding or generate with AI.
```

```
Start coding or generate with AI.
```